

Projeto Extra

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Projetos

Este projeto é estritamente individual. Seleciona **um** dos seguintes projetos. Todos os projetos serão alojados no **GitHub**, utilizando o [GitHub Classroom](#). Consulta [aqui](#) os detalhes.

O repositório deve conter todos os scripts relevantes, ficheiros de configuração e um ficheiro README .md com instruções detalhadas sobre como fazer o deploy do projeto. Deve também conter um relatório do projeto em formato PDF.

Este é o **projeto extra**, desenhado para integrar as diversas competências adquiridas ao longo do semestre (Shell Scripting, Docker, Python, Análise de Dados e Tecnologias Web) integrando os conceitos do **Projeto 01** e **Projeto 02** para estudantes em fase de avaliação de recurso.

Este é um projeto com a duração de três semanas (prazo limite **23/07/2026**). Tens até ao final desta semana para notificar o teu professor (via e-mail) sobre o tema escolhido (a lista de temas pode ser consultada [aqui](#)).

Não te esqueças de contactar o teu professor em caso de dúvidas. Instruções adicionais podem ser adicionadas.

Temas

1. Plataforma de Monitorização de Frotas e Tráfego de Cidades Inteligentes

- **Descrição:** Implementa um pipeline de monitorização de trânsito de cidades inteligentes. Múltiplos contentores de simulação de veículos (scripts em Python) publicam coordenadas GPS, velocidade e telemetria de combustível via **MQTT**. Um *broker* **Mosquitto** encaminha esta telemetria. Um serviço de recolha **FastAPI** subscreve

os tópicos MQTT, valida os registos recebidos e armazena-os numa base de dados **PostgreSQL**. O **Grafana** é implementado para visualizar rotas de veículos e tendências de velocidade média em tempo real. Um contentor **Caddy** (ou Nginx) serve como proxy inverso de cache em frente ao painel do Grafana. Um **script Bash de backup** corre periodicamente via cron dentro de um contentor de backup dedicado para exportar a base de dados PostgreSQL, comprime-a num ficheiro `.tar.gz` com data/hora e roda os arquivos antigos (mantendo apenas os 7 mais recentes).

- **Competências Chave:** Docker Compose (rede bridge multi-contentor), MQTT (Mosquitto), Persistência SQL (PostgreSQL com volumes persistentes), Proxy Inverso e Caching (Caddy/Nginx), Scripting Bash e Automação (cron, tar, rotação de ficheiros), Grafana.

2. Espaço de Trabalho de Documentos Partilhados Colaborativo com Análítica de Versões

- **Descrição:** Cria um bloco de escrita colaborativo e partilhado em tempo real. A aplicação web permite que múltiplos utilizadores escrevam Markdown, sincronizando as alterações de texto entre clientes instantaneamente via **WebSockets**. O backend usa **Redis Pub/Sub** para gerir a concorrência na difusão de caracteres e armazena o histórico de todas as versões do documento numa base de dados **PostgreSQL**. Um contentor *offline* separado e personalizado executa um **script Python** utilizando **Pandas** ou **Polars** para ler a base de dados periodicamente, calcular métricas chave (como contagem de palavras, adições de caracteres e percentagens de contribuição por estudante/autor) e guardar um relatório analítico resumido em markdown. Um **script daemon Bash** corre no host (ou num contentor privilegiado) para monitorizar o tamanho do diretório de armazenamento, registar o uso de recursos do contentor de base de dados e registar batimentos de atividade (*heartbeats*) num log de sistema. Os programadores devem usar uma configuração de **Dev Container** (Dockerfile configurando ferramentas de depuração e linters) para o desenvolvimento local.
- **Competências Chave:** WebSockets, Redis Pub/Sub, PostgreSQL, Dev Containers (Dockerfile), Análítica de Dados em Python (Pandas/Polars), Scripting Bash Daemon (monitorização de integridade e disco), Docker Compose.

3. Motor de Ingestão de Poluição do Ar e Saúde Pública Global

- **Descrição:** Cria um painel geográfico que visualiza métricas de poluição do ar global. Um **serviço Python** consulta periodicamente

uma **API REST externa** (ex: API OpenAQ) para recolher densidades de poluentes atmosféricos (PM2.5, NO2, SO2) em 5 capitais internacionais pré-selecionadas, persistindo os dados brutos numa base de dados. Um **script Python** de análise separado utilizando **Pandas** corre para detetar violações dos limites de segurança, calcular desvios padrão (volatilidade) e exportar dados JSON estruturados para um volume partilhado. Um servidor web **Nginx** aloja um portal estático com um **mapa interativo Leaflet.js** que lê os dados JSON e exibe marcadores coloridos com base nas camadas de gravidade da qualidade do ar. Um **script Bash** corre com um temporizador para verificar a disponibilidade da API externa (usando `curl`), inspecionar os limites de taxa REST (*rate-limits*) e anexar logs de estado.

- **Competências Chave:** APIs REST Externas, Análise de Dados (Pandas), Visualização de Mapas (Leaflet.js), Servidor Web Nginx, Automação Bash (verificação de API com `curl`), Docker Compose.

4. Cofre de Telemetria e Desencriptador de IoT Industrial Seguro

- **Descrição:** Implementa um cofre seguro de logs industriais que automatiza a recolha de metadados e a proteção criptográfica. Um serviço web **FastAPI** gere uploads autenticados de logs de máquinas (ficheiros CSV ou brutos) e consulta automaticamente uma base de dados para extrair metadados das máquinas com base nos cabeçalhos dos identificadores. Quando um novo ficheiro de log é carregado, um **script Bash** em segundo plano atuando como observador de diretório (*directory watcher*) deteta o ficheiro, verifica a sua integridade (hashing SHA256), encripta-o usando **GnuPG** ou **OpenSSL** com uma frase-chave/chave de máquina segura e elimina o original não encriptado. O sistema deve impor permissões estritas de **ficheiros Linux** (`chmod 600/chown` no volume) para que apenas o agente de encriptação possa ler os ficheiros brutos. Uma interface web exibe o catálogo de metadados de máquina e permite que operadores autorizados solicitem, desencritem e inspecionem os logs de máquina dinamicamente.
- **Competências Chave:** Conceitos de Segurança (encriptação GnuPG/OpenSSL, hash SHA256), Permissões do Sistema Linux (`chmod/chown`), Uploads FastAPI, Persistência em Base de Dados, Extração de APIs REST Externas, Monitorização de Diretórios em Bash.

5. Pipeline de CI/CD de Relatórios de Análítica Desportiva Baseado em Base de Dados

- **Descrição:** Desenha um pipeline automatizado de Integração Contínua que compila relatórios analíticos desportivos a partir de regis-

tos da base de dados. Uma aplicação multi-contentor armazena estatísticas de jogos ao vivo ou métricas de equipas numa base de dados **PostgreSQL**. Um **script Bash** monitoriza uma vista de dados ou pasta de logs do PostgreSQL para atualizações. Quando uma alteração é detetada, ele despoleta um contentor **Docker** personalizado equipado com **Pandoc**, **LaTeX** e **Python**. Este contentor executa um script Python que corre consultas, analisa a base de dados desportiva usando **Pandas**, gera gráficos estatísticos (PNGs), atualiza um modelo dinâmico em **Markdown** e compila-o via **Pandoc** num relatório final em PDF. O PDF compilado é copiado automaticamente para um volume servido por um servidor web **Caddy**, permitindo aos treinadores descarregar o relatório atlético mais recente.

- **Competências Chave:** Dockerfiles Customizados (Pandoc + LaTeX + Python), Automação Bash (monitorização de base de dados/pastas), Desenho de Gráficos em Python (Matplotlib/Seaborn), Compilação de PDF com Pandoc, Bases de Dados SQL, Servidor Web Caddy/Nginx.

6. Sistema de Inventário de Ativos de Armazém com Wiki Técnica

- **Descrição:** Cria um pipeline de rastreio de armazém inteligente apoiado por documentação técnica interativa. Simuladores publicam sinais de leitura de código de barras e movimentos de inventário via **MQTT** para um *broker* **Mosquitto**. Um serviço de recolha **FastAPI** armazena esta telemetria no **PostgreSQL**. O **Grafana** é implementado para exibir tendências de inventário e despoletar alertas quando os níveis de stock caem abaixo de limites definidos. Para documentar a arquitetura técnica, um contentor **BookStack** ou **DokuWiki** é implementado juntamente com o sistema, pré-populado através de um volume montado com pelo menos 5 páginas markdown detalhando subredes de rede de contentores, esquemas de bases de dados de inventário e instruções de configuração. Um **script Bash** corre periodicamente para verificar o uso de CPU e memória de todos os contentores, registrando a pegada de recursos.
- **Competências Chave:** MQTT (Mosquitto), PostgreSQL, Grafana, Implementação de Wiki (BookStack/DokuWiki), Volumes e Redes Docker, Telemetria de Recursos com Bash.

Acesso ao GitHub Classroom

Aqui estão instruções detalhadas para aceder ao GitHub Classroom. A maioria dos estudantes pode saltar vários passos, visto que estes foram concluídos em projetos anteriores.

1. Juntar-se ao Trabalho e Criar a Tua Equipa

1. **Aceder ao link:** Ir para [aqui](#)
 2. **Encontrar o teu nome:** Selecionar o teu nome da lista de estudantes. > **Não encontras o teu nome?** Todos os nomes registados no PACO foram adicionados. Se o teu estiver em falta, por favor contacta o [Prof. Mário Antunes](#).
 3. **Criar uma equipa:** Seguir esta estrutura exata de nomenclatura: [nmec]_project04
 - (Exemplo: 132745_project04)
-

2. Acesso à Organização e Repositório

1. **Aceitar o convite por email:** Depois de se juntar a uma equipa, todos os membros receberão um convite por email para se juntarem à organização GitHub detiuaveiro.
 2. **Deves aceitar este convite** antes de poderes continuar.
 3. **Atualizar a página:** Voltar à página do GitHub Classroom e atualizá-la.
 4. **Verificar o acesso:** Deves agora ver e ter acesso ao repositório de trabalho da tua equipa.
-

3. Configurar uma Chave SSH para Acesso

Isto permitir-te-á clonar e enviar alterações para o repositório a partir da tua linha de comandos sem teres de introduzir a tua palavra-passe de cada vez.

1. **Verificar se existe uma chave SSH:** Abre o teu terminal e executa este comando:

```
cat ~/.ssh/id_ed25519.pub
```
2. **Gerar uma chave (se necessário):**
 - Se vires uma chave (a começar por ssh-ed25519...), copia a linha inteira e salta para o passo 3.
 - Se vires um erro como "No such file or directory," executa o seguinte comando para criar uma nova chave:

```
ssh-keygen -q -t ed25519 -N ''
```
 - Depois de gerada, corre `cat ~/.ssh/id_ed25519.pub` novamente para ver a tua nova chave e copiá-la.

3. Adicionar a chave à tua conta GitHub:

- Vai às **Definições** do teu GitHub.
- No menu esquerdo, clica em **SSH and GPG keys**.
- Clica no botão **New SSH key**.
- Dá-lhe um **Título** (ex: "O Meu Portátil UA").
- Cola a chave que copiaste no campo **Key**.
- Garante que o "Key type" está definido para **Authentication Key**.
- Clica em **Add SSH key**.

4. Autorizar a chave para SSO:

- Depois de adicionar a chave, encontra-a na tua lista na mesma página.
- Clica em **Configure SSO**.
- Seleciona a organização **detiuaveiro**, preenche os teus dados de login e concede acesso.